

Priority Queues

Priority Queue ADT

- Insertion, deletion, and delete_highest_priority
- Priority associated with the object as the key of that object
- insert(k, x): inserts an entry with key k and object x
- removeMin() (deleteMin or extractMin): removes and returns the smallest key entry
- min(): returns, but do not remove, an entry with smallest key
- size(), isEmpty()

Total Order Relations

滿足以下三大離散數學定律，確保 key 可以被比較

1. Reflexive property: $x \leq x$
2. Antisymmetric property: $x \leq y \wedge y \leq x \Rightarrow x = y$
3. Transitive property: $x \leq y \wedge y \leq z \Rightarrow x \leq z$

Entries

A key-element pair

Representing a Priority Queue

Operation	Unsorted List	Sorted List	Heap
Insert	$\Theta(1)$	$\Theta(n)$	$\Theta(\log n)$
RemoveMin / min	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$

Implementing a Priority Queue with a List

With a unsorted list

Main Idea : **Selection-sort**

→ Each insert will take $O(1)$, n element will take $O(n)$

→ Each remove will take $O(n)$, n element will take $O(n^2)$

With a sorted list

Main Idea : **Insertion-sort**

→ Each insert will take $O(n)$, n element will take $O(n^2)$

→ Each remove will take $O(1)$, n element will take $O(n)$

With a heap

→ Each insert will take $O(\log n)$, n element will take $O(n \log n)$

→ Each remove will take $O(\log n)$, n element will take $O(n \log n)$

Heaps

Heap property

A min heap: The value of the node is **smaller** than its children

A max heap: The value of the node is **bigger** than its children

Since heap is a **complete binary tree** and one can use an **array** to represent it

Construct a heap

1. Insert each element one by one

$O(\log n)$ for each insertion, $O(n \log n)$ for total operation

2. Build a heap by an array

- a. Make the array an BST tree

- b. Do heapify to turn BST to a heap

針對 array 中的 Index $\lfloor \frac{n}{2} \rfloor \sim 0$ 依序進行 heapify 操作，後面不用，因為已經是底了

heapify → Min Heap 中選最小的子節點 SWAP

Max Heap 中選最大的子節點 SWAP

互換後，要跟蹤剛才選擇的節點繼續作 **heapify**

